CSC 360, Software Development with AI
Project #1

For this project, you will create an Eastern Registration System by modifying and expanding the code provided.

Current Project structure

- *cache* – folder used to cache schedule data, including available terms and course schedules by subject
- *demo/docs* – location of .puml files
- *demo/src/main/java/com/example* – location of source code
    - *EasternRegistration.java* – the main program (currently includes a demo of how to use an *EasternSchedule* object)
    - *Course.java* – defines a course record (e.g., information about CSC 360-02 for Fall 2026)
    - *EasternSchedule.java* – Object used to retrieve and query information from Eastern's schedule of classes
    - *Term.java* – defines a term record (e.g., corresponding to Fall 2026)
- demo/pom.xml – configuration file for Maven
- .gitignore – files and folders to be ignored by git (you may add to this but should not delete anything from the original version)
- README.md – README file for Github repository

Apache Maven

Apache Maven is a build automation and project management tool for Java applications. We will talk about Maven in a little more detail after this project. The Project Object Model (POM) file (pom.xml) tells Maven everything it needs to know about a project, such as its dependencies. In this case, we have a dependency on Jackson Databind (for converting JSON objects to Java objects). Without a project management tool, we would need to download the corresponding JAR files and link these to the project. Using a tool such as Maven makes this process easier.

Occasionally it will be useful to *clean* your project, which removes the target directory including all bytecode. This can be done from the sidebar, by clicking the Play button next to Maven → demo →

Lifecycle → clean. To run code after cleaning, you will need to recompile it. This can be done by clicking the Play button next to Maven → demo → Lifecycle → compile. You will now be able to run code using the Play button or clicking on the Run command above *main* as we have been doing in class.

Project Requirements

1. You must use Github to manage your project (Note: you may make your repository private if you prefer, as long as you invite me as a collaborator). [15 points]
    a. Initialize your repository from within VS Code by opening the command palette and selecting Git: Initialize Repository. Your initial commit should include all original files provided except for the *cache* files, with no changes made. Make sure to include .gitignore and .vscode. Your initial commit should trigger a Github action that compiles your code. You should see a green check mark after your initial commit. If this is not the case, then let me know.
    b. Following best practices, create dedicated branches corresponding to features as you modify and expand the code. For example, you can create a *main_menu* branch to complete 3(a) and 3(b) below, and then create a *registration* branch to complete 3(c) and 3(d). After accepting pull requests, make sure to use *git pull* (or the pull icon from VS Code) to pull in changes from Github to your local repository.
    c. Commits must be descriptive. If you use AI, include your prompt in the commit and specify whether any manual changes were needed. It is good practice to commit and push often. A commit should represent a single, logical change (such as a bug fix, addition of a class, etc). Click here for an overview of proper git comment formatting.
2. Update demo/docs/easternRegistration.puml to match your program design. You should have one puml file containing all classes (except for *EasternRegistration*). In particularl, you will need to add *MainMenu* and *Student* classes. For efficiency with using AI, I recommend creating separate puml files first for each new class and then asking the AI to generate the class based on the single file. However, your final *easternRegistration.puml* file should include all classes and records. [10 points]

3. Create a *MainMenu* class that allows the user to do the following.

   Note: Do not modify any* private *methods in EasternSchedule. You may modify public methods and may add new public/private methods. To add implement additional search criteria, start with the results from *queryCourseBySubject*, then filter these results as needed.

   a. Display all courses for a specified subject [10 points]
      i. Course information must include subject, number, title, credits, days, and times in a reasonable format, e.g.
      CSC 360-02, 3 credits, Software Development with AI, MWF, 2:00 – 3:15 PM
      ii. You may include more information (such as instructor, location, enrollment, etc)
      iii. Note that some courses have multiple meetings, such as BIO 230
   b. Display all courses for a subject with additional search criteria: [10 points]
      i. Subject and number (e.g., CSC 360) (required)
      ii. Only open courses (optional)
      iii. Instructor (optional)
      iv. Exclude day of the week (e.g.., no courses on Tuesdays) (optional)
   c. Allow a student to add or drop classes (see below)
   d. Display the schedule of classes for the student [10 points]

4. Create a *Student* class that allows a student to add and drop courses
   a. Create a student by specifying an ID, a first name, and last name. [10 points]
   b. This class has a *static final int* MAX_CREDITS field which is set to 17 (but you can lower this for testing).
   c. Allow the student to add one or more classes, *up to* MAX_CREDITS classes. Do not allow the student to register for a course that is already full. Do not allow the student to register for two courses if there is a time conflict. [15 points]
   d. Allow the student to drop all classes, or one or more specific classes [10 points]
   e. Optional: Save the student's schedule to a file using JSON serialization (see section 3.1 of https://www.baeldung.com/jackson-object-mapper-tutorial). Note the Student class must have a default constructor and getters and setters for all fields for this to work.

5. Include JavaDoc comments. The documentation (html files) should not be included with your code, but you should test that these can be generated using the following command:

   ```
   mvn -f demo/pom.xml javadoc:javadoc
   ```

   Documentation will be generated in demo/target/reports/apidocs/ with *index.html* being the main page. [10 points]

6. Additional requirements (up to 10-point deduction for each if not met)
   a. The only *main* method should be in *EasternRegistration.java* (you can add *main* methods in other classes for testing, but these should be removed prior to your final submission. Running this code should open the main menu which prompts the user to select an option (from (3) above). The main method should look something like this:

      ```
      MainMenu menu = new MainMenu();
      menu.open();
      ```

   b. Your code must compile (using Maven) and run
   c. Your github repository does not include any extraneous files, including additional cache files.
   d. Follow best practices for Java coding
      i. Use informative variable, object, and method names
      ii. Class instance variables should be *private*
      iii. A class should represent one single concept or piece of functionality, and should have only one reason to change
   e. Have fun!